Detecting Repeated SSH Connections via IP Address Reputation

Supakjeera Thanapaisal	SXT240029
Timothy Sweet	TRS200004
Sherry Cherniavsky	SXC210068

GitHub: https://github.com/nthanapaisal/ssh-auth-attempt-reputation-based-ban

Demo: https://drive.google.com/file/d/1mPqFKtp0mOVPOBIdwFrvvuMNmrOMHcpk/

April 2026

Contents

Introduction

A lightweight security system that detects and blocks repeated SSH authentication attempts on a Linux VPS using Autonomous System (AS) reputation. The system monitors authentication logs, evaluates attacker reputation via AbuseIPDB, and dynamically updates `nftables` firewall rules to block malicious connections.

The key advantage over tools like Fail2ban is that banning decisions are informed by *global* reputation data rather than only locally-observed failures. An IP already known to be malicious can be blocked on its very first attempt against the local server, without waiting for a failure threshold to be crossed. A local SQLite cache keeps API usage efficient — the external service is only consulted when the information would genuinely be new.

Three components cooperate to achieve this: a **log listener service** that tails the systemd journal for SSH failure events, a **reputation service** that decides whether a given IP should be blocked, and an **nftables firewall configuration** that enforces those decisions at the kernel level.

Background and Motivation

The Problem

SSH servers exposed to the internet are constant targets for automated brute-force attacks. Even failed attempts flood logs, consume CPU handling rejected handshakes, and make it harder to spot legitimate anomalies. At high enough volumes, this traffic can degrade SSH availability for real users.

Existing Defenses

Tools like Fail2ban and DenyHosts ban IPs after a locally-observed failure threshold is crossed. They are purely reactive — an IP that has been attacking other servers for months still gets several free attempts locally before any ban is applied. Static blocklists offer proactive blocking but require constant manual maintenance and go stale quickly.

SSH servers can and should be configured with strong authentication — ideally key-based only, with password auth disabled — to prevent any attempt from succeeding. However, blocking repeated failed attempts at the firewall level is still valuable independent of authentication strength.

Threat Model

The threat modeled here is **denial-of-service via SSH flooding**: high volumes of repeated, failed authentication attempts. Successful attacker login is out of scope — that is prevented by key-based authentication. Two concrete harms motivate blocking this traffic even when it cannot succeed:

1. Repeated failed attempts clog the system log, making it harder to find real entries and troubleshoot SSH-related issues.
2. At sufficient volume, the connection overhead can consume enough resources that SSH becomes slow or difficult to reach for legitimate users.

By blocking traffic from known-bad IPs at the kernel level, the system prevents `sshd` from ever processing those connections.

Design

Architecture

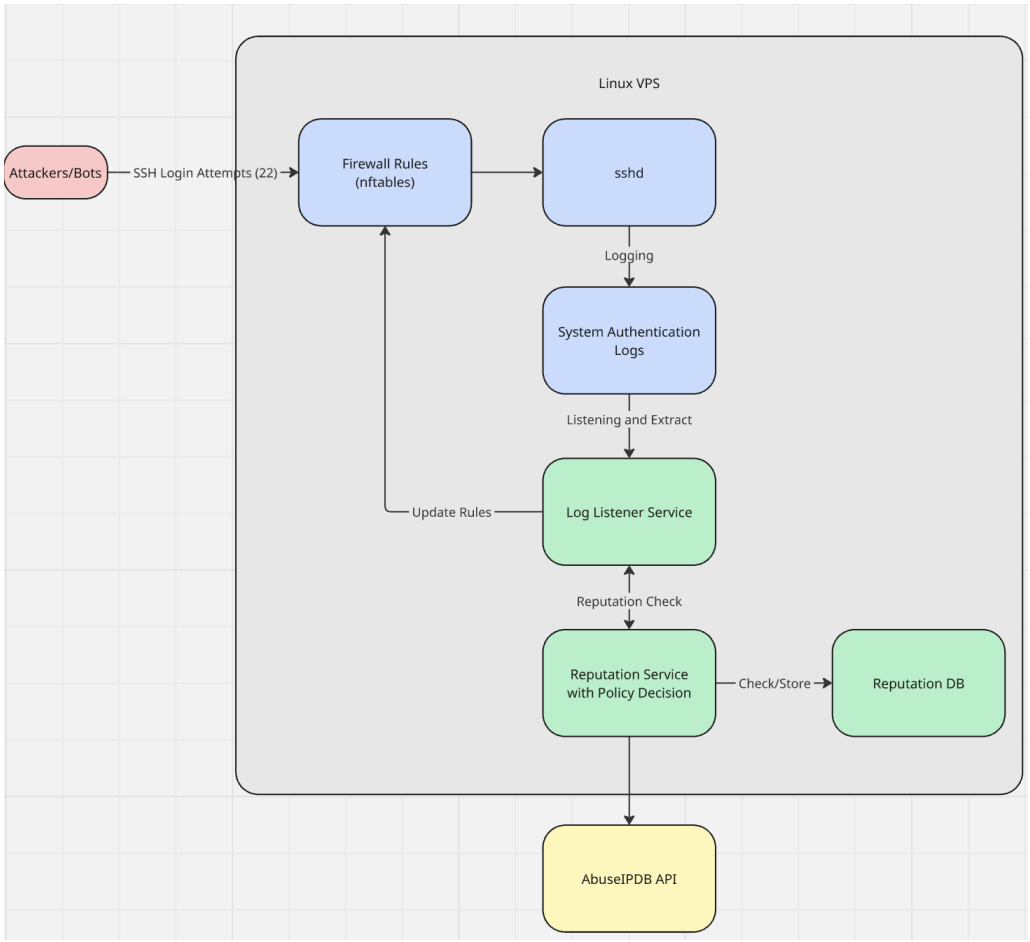

Figure 1: High-level architecture diagram.

Incoming SSH attempts pass through `nftables`. If not already banned, `sshd` processes the attempt and writes a log entry to the systemd journal. The listener watches for failure log entries, extracts the source IP, and passes it to the reputation service. The reputation service checks a local SQLite cache, calls AbuseIPDB if needed, and returns a block-or-allow decision. A block decision causes the listener to insert the IP into an `nftables` set with a 12-hour expiration.

Components

File	Responsibility
`listener_service.py`	Monitors `ssh.service` journal entries in real time. Extracts source IPs from failure log messages and invokes the reputation service.
`reputation_service.py`	Validates IPs, checks the SQLite cache, queries AbuseIPDB when needed, and returns a block decision.
`ssh_auto_ban-default.nft`	Defines `nftables` sets for banned IPv4/IPv6 addresses with a drop rule on the input chain.

Component Design

Listener Service (`listener_service.py`)

On startup, the listener loads `ssh_auto_ban-default.nft` via `nft -f` to initialize the firewall sets, then opens a `systemd.journal.Reader` filtered to `ssh.service`. It seeks to the journal tail (ignoring historical entries) and registers the file descriptor with `select.poll()`, blocking until new entries arrive.

Each new entry's `MESSAGE` field is matched against four patterns:

- `Failed password for [invalid user] <user> from <ip>` — password auth failure.
- `Unable to negotiate with <ip>` — cipher/key-exchange mismatch; typical of automated scanners probing with non-standard parameters.
- `banner exchange: Connection from <ip>` — low-level protocol failure before authentication; characteristic of scan tools.
- `Invalid user <user> from <ip>` / `Connection closed by authenticating user` — only matched when `SSH_HAS_PASS_AUTH_DISABLED = True`. When password auth is enabled, these events overlap with the failed password pattern and would double-count the same connection.

If a match is found, the IP is passed to `handle_bad_ip`, which calls the reputation service. On a block decision, `ban_bad_ip` determines IPv4 vs. IPv6 using Python's `ipaddress` module and inserts the address into the appropriate `nftables` set:

```
nft add element inet ssh_auto_bans <set> { <ip> timeout 12h }
```

The 12-hour timeout is handled natively by the kernel — no external cleanup process is needed.

Reputation Service (`reputation_service.py`)

IP Validation

Before any cache or API call, the IP is validated with `ipaddress.ip_address()`. Non-global addresses (private, loopback, link-local) immediately return `False`. This prevents the system from acting on log messages that reference internal addresses and ensures nothing private is sent to the external API.

Caching and Decision Logic

AbuseIPDB provides an *abuse confidence score* (0–100) reflecting community-reported malicious activity. Scores above 80 result in a block. Rather than querying the API on every connection, the service uses a tiered cache to decide when a fresh lookup is warranted:

1. **Not in DB** — query AbuseIPDB, store result, return decision.
2. **In DB, already blocked** — return `True` immediately; increment counter.
3. **In DB, not blocked, cache fresh** ($\leq$1 hour since last check) — return `False`; increment counter.
4. **In DB, not blocked, cache expired, count $\leq$ 20** — return `False`; increment counter. The IP is not appearing frequently enough for a fresh lookup to matter.
5. **In DB, not blocked, cache expired, count $>$ 20** — the IP is persisting despite not being blocked. Query AbuseIPDB for a fresh score, update DB, return new decision.

Database Schema

```
CREATE TABLE IF NOT EXISTS ip_rep_db (
    ip           TEXT    PRIMARY KEY,
    score        INTEGER NOT NULL,
    checked_at   INTEGER NOT NULL,  -- Unix timestamp of last API call
    num_connects INTEGER NOT NULL,  -- connections since last API call
    is_blocked   INTEGER NOT NULL
);
```

nftables Configuration (`ssh_auto_ban-default.nft`)

Defines an `inet` table (covering both IPv4 and IPv6) with two sets: `banned_v4` and `banned_v6`. Both use `flags timeout` so that individual entries expire automatically. A single input chain with default `accept` policy drops and logs any packet whose source address appears in either set.

```
table inet ssh_auto_bans {
    set banned_v4 { type ipv4_addr; flags timeout }
    set banned_v6 { type ipv6_addr; flags timeout }

    chain input {
        type filter hook input priority 0; policy accept;
        ip  saddr @banned_v4 log prefix "SSH autoban: " drop
        ip6 saddr @banned_v6 log prefix "SSH autoban: " drop
    }
}
```

Program Flow Summary

1. Monitors `ssh.service` logs through `systemd.journal`.
2. Extracts source IPs from failed or suspicious SSH log messages.
3. Sends the IP to the reputation service.
4. Validates that the IP is a real global/public address.
5. Checks the local SQLite cache.
6. Calls AbuseIPDB only if:
   6.1 the IP is not in cache, or
   6.2 the cache is expired and the IP has exceeded the connection threshold.
7. Stores or updates the IP record in SQLite.
8. Returns whether the IP should be blocked.
9. If blocked, adds the IP to an `nftables` ban set with a 12-hour timeout.

The goal is to reduce repeated malicious SSH connection attempts by using IP reputation data together with a local cache, rather than calling the external API on every connection.

Evaluation

Experimental Setup

The test was conducted on a cloud VPS with port 22 (`sshd`) exposed to the internet. It was measured over a period from `2026-04-28 15:33:33 UTC` to `2026-04-30 08:36:55 UTC` (approximately 2 days and 5 hours). During this time:

- Existing `fail2ban` protections were disabled.
- `sshd` password authentication was enabled.
- No legitimate login attempts were made.

Observed Traffic

Metric	Count	Unique IPs
Connections reaching `sshd`	3,379	203
Connections blocked by firewall	20,609	163
Total observed attempts	23,988	203

Table 1: Summary of observed SSH connection attempts

Effectiveness of Filtering

From the observed data:

- Only **16.4%** of total connection attempts reached `sshd`.

- The firewall blocked **83.6%** of all attempts before they reached the service.
- **80.2%** of unique IP addresses were blocked after their first attempt, preventing repeated probing.

Discussion

As we intended, the firewall rules eliminate the vast majority of connection attempts, with 80.2% of all IPs that attempt to connect to us being blocked on the very first try, and 83.6% of total connections never reach `sshd` at all. This indicates that utilizing crowd-sourced IP reputation info is a useful layer of defense to reduce connection attempts and load on `sshd`, in addition to further layers such as traditional fail2ban and of course, strong authentication.

We are unable to directly evaluate false positives because legitimate IP addresses are not listed in AbuseIPDB, which is an external system. Additionally, we avoid adding legitimate IPs to the database for testing, as this would interfere with the integrity of the system. However, in practical deployments, false positives can be mitigated through mechanisms such as firewall allowlists ("firewall rules"), especially in environments where the set of legitimate IPs is small and well-defined.

For false negatives, the objective of our project is not to block all malicious IPs, but rather to block IPs that are identified by AbuseIPDB. Our results show that this approach successfully blocks approximately 82% of malicious IPs. Improving coverage beyond this point depends on the quality and completeness of the external database. Therefore, the primary way to reduce false negatives is to contribute additional threat intelligence back to AbuseIPDB and rely on improvements made by the broader community.

Installation

Prerequisites: Python 3.8+, SQLite, `nftables`, `systemd`, and an AbuseIPDB API key (https://www.abuseipdb.com/)

```
# Set environment variables
export ABUSEIPDB_API_KEY="your_key_here"
export REPUTATION_DB_PATH="path/to/reputation.sqlite"

# Clone and install
git clone https://github.com/nthanapaisal/ssh-auth-attempt-reputation-based-ban.git
cd ssh-auth-attempt-reputation-based-ban
pip install -r requirements.txt

# Run (requires root for nftables)
sudo python listener_service.py
```